

Развертывание Platforma на Beget

Пошаговая инструкция по загрузке проекта на VPS/VDS Beget, настройке Docker, PostgreSQL/PostGIS, Redis, S3/MinIO, Nginx, SSL и первичной проверке платформы.

Проект: `/Users/nick/Documents/platforma` . Дата подготовки: 12 мая 2026.

Важно. Этот проект нельзя просто загрузить в `public_html` обычного shared-хостинга. Ему нужен постоянно работающий NestJS API, PostgreSQL с расширением PostGIS, Redis, S3-совместимое хранилище и сборка через `pnpm`. Рабочий вариант для Beget - VPS/VDS в Beget Cloud с Docker.

0. Что есть в проекте

- `apps/web` - Vite + React + Tailwind frontend.
- `apps/api` - NestJS API, Prisma, auth/RBAC, объекты, файлы, карта, WordPress import.
- `packages/shared` - общие TypeScript-типы.
- `tools/wp-import` - CLI-инструмент импорта данных из WordPress.
- `docker-compose.yml` - локальный запуск PostgreSQL/PostGIS, Redis, MinIO, API и web.

1. Подготовьте Beget VPS

1. В панели Beget создайте VPS/VDS, а не обычный виртуальный хостинг.
2. В маркетплейсе выберите образ **Docker**. Он устанавливает Ubuntu, Docker CE и Docker Compose.
3. Рекомендуемая минимальная конфигурация: 2 CPU, 4 GB RAM, 40 GB NVMe. Для тяжелого импорта WordPress лучше больше.
4. Создайте DNS-записи:
 - `platform.example.ru` A-record на IP сервера.
 - `api.platform.example.ru` A-record на IP сервера.

В примерах ниже замените `SERVER_IP`, `platform.example.ru`, `api.platform.example.ru`, пароли и секреты на свои значения.

2. Подключитесь к серверу

```
ssh root@SERVER_IP
```

Проверьте, что Docker работает:

```
docker run hello-world
docker compose version
```

3. Загрузите проект на сервер

Вариант А: через Git

```
mkdir -p /opt
cd /opt
git clone YOUR_REPOSITORY_URL platforma
cd /opt/platforma
```

Вариант В: через rsync с локальной машины

```
rsync -av --exclude node_modules --exclude dist --exclude .git \
/Users/nick/Documents/platforma/ root@SERVER_IP:/opt/platforma/
```

После загрузки на сервере:

```
cd /opt/platforma
```

4. Создайте production .env

```
cd /opt/platforma
cp .env.example .env
nano .env
```

Минимальный пример значений:

```
COMPOSE_PROJECT_NAME=platforma

POSTGRES_DB=platforma
POSTGRES_USER=platforma
POSTGRES_PASSWORD=STRONG_POSTGRES_PASSWORD
DATABASE_URL=postgresql://platforma:STRONG_POSTGRES_PASSWORD@postgres:5432/platforma?schema=public

REDIS_URL=redis://redis:6379

MINIO_ROOT_USER=platforma_admin
MINIO_ROOT_PASSWORD=STRONG_MINIO_PASSWORD
MINIO_BUCKET=platforma
S3_ENDPOINT=http://minio:9000
S3_PUBLIC_ENDPOINT=http://minio:9000
S3_REGION=us-east-1
S3_ACCESS_KEY_ID=platforma_admin
S3_SECRET_ACCESS_KEY=STRONG_MINIO_PASSWORD

API_PORT=3000
WEB_PORT=5173
VITE_API_URL=https://api.platform.example.ru
WEB_ORIGIN=https://platform.example.ru

JWT_ACCESS_SECRET=GENERATE_LONG_RANDOM_SECRET
JWT_REFRESH_SECRET=GENERATE_ANOTHER_LONG_RANDOM_SECRET
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL_DAYS=30
REFRESH_COOKIE_NAME=platforma_refresh_token

VITE_YANDEX_MAPS_API_KEY=
```

Секреты можно сгенерировать командой:

```
openssl rand -hex 32
```

Не оставляйте значения `change-me-access-secret`, `change-me-refresh-secret` и простые пароли из примера. Не публикуйте файл `.env` в репозиторий.

5. Создайте production docker-compose

В текущем `docker-compose.yml` наружу публикуются порты PostgreSQL, Redis, MinIO, API и web. Для production лучше закрыть базу, Redis и MinIO от интернета, а API/web отдавать через Nginx.

```
cp docker-compose.yml docker-compose.prod.yml
nano docker-compose.prod.yml
```

В `docker-compose.prod.yml`:

- У сервисов `postgres`, `redis`, `minio` уберите блоки `ports`.
- У сервиса `api` оставьте локальную публикацию:

```
ports:
  - "127.0.0.1:3000:3000"
```

- У сервиса `web` оставьте локальную публикацию:

```
ports:
  - "127.0.0.1:5173:5173"
```

Это не меняет код проекта, но защищает внутренние сервисы от прямого доступа извне.

6. Запустите проект

```
cd /opt/platforma
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
```

API-контейнер при старте выполнит Prisma migrations через команду из `apps/api/Dockerfile`.

Проверьте здоровье API:

```
curl http://127.0.0.1:3000/health
```

Ожидаемый ответ:

```
{
  "status": "ok",
  "database": "ok",
  "postgis": true
}
```

7. Создайте первого admin-пользователя

```
docker compose -f docker-compose.prod.yml exec \
-e ADMIN_EMAIL=admin@example.ru \
-e ADMIN_NAME="Platform Admin" \
-e ADMIN_PASSWORD="STRONG_ADMIN_PASSWORD" \
api npm --filter @platforma/api db:seed
```

После первого входа замените пароль администратора. Значения `ADMIN_PASSWORD` нужны только для первичного seed.

8. Настройте Nginx и SSL

```
apt update
apt install -y nginx certbot python3-certbot-nginx
```

Создайте конфиг для frontend:

```
nano /etc/nginx/sites-available/platforma-web
```

```
server {
    server_name platform.example.ru;

    location / {
        proxy_pass http://127.0.0.1:5173;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Создайте конфиг для API:

```
nano /etc/nginx/sites-available/platforma-api
```

```
server {
    server_name api.platform.example.ru;

    client_max_body_size 60m;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Включите сайты и выпустите SSL:

```
ln -s /etc/nginx/sites-available/platforma-web /etc/nginx/sites-enabled/platforma-web
ln -s /etc/nginx/sites-available/platforma-api /etc/nginx/sites-enabled/platforma-api
nginx -t
systemctl reload nginx
certbot --nginx -d platform.example.ru -d api.platform.example.ru
```

9. Если используете Beget S3 вместо MinIO

1. В панели Beget создайте S3 bucket вручную.
2. Скопируйте из панели: URL, имя bucket, Access key и Secret key.
3. В `.env` замените S3-настройки:

```
S3_ENDPOINT=https://s3.ru1.storage.beget.cloud
S3_PUBLIC_ENDPOINT=https://s3.ru1.storage.beget.cloud
S3_REGION=ru1
S3_ACCESS_KEY_ID=ACCESS_KEY_FROM_BEGET
S3_SECRET_ACCESS_KEY=SECRET_KEY_FROM_BEGET
MINIO_BUCKET=YOUR_BEGET_BUCKET_NAME
```

Не указывайте в `S3_PUBLIC_ENDPOINT` URL уже с bucket. Код проекта сам добавляет `MINIO_BUCKET` к endpoint. Если вставить URL с bucket, ссылки могут получить дубль bucket.

Перезапустите API после изменения `.env` :

```
docker compose -f docker-compose.prod.yml up -d --build api
```

10. WordPress import

Если нужен импорт из WordPress, заполните в `.env` :

```
WP_DB_HOST=
WP_DB_PORT=3306
WP_DB_USER=
WP_DB_PASSWORD=
WP_DB_NAME=
WP_TABLE_PREFIX=wp_
WP_POST_TYPE=nedvizhimosts
WP_IMPORT_LIMIT=
WP_UPLOADS_HOST_PATH=/absolute/path/to/wp-content/uploads
WP_UPLOADS_PATH=/wp-uploads
```

Сначала запускайте preview:

```
docker compose -f docker-compose.prod.yml exec api npm wp-import:preview
```

Только после проверки отчета запускайте полный импорт:

```
docker compose -f docker-compose.prod.yml exec api npm wp-import:run
```

11. Обновление проекта после изменений

Если проект загружен через Git

```
cd /opt/platforma
git pull
docker compose -f docker-compose.prod.yml up -d --build
```

Если проект загружен через rsync

Повторите загрузку с локальной машины, затем на сервере:

```
cd /opt/platforma
docker compose -f docker-compose.prod.yml up -d --build
```

12. Диагностика

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f api
docker compose -f docker-compose.prod.yml logs -f web
docker compose -f docker-compose.prod.yml logs -f postgres
docker compose -f docker-compose.prod.yml logs -f minio
curl http://127.0.0.1:3000/health
```

13. Финальный чеклист

- ☐ VPS создан именно с Docker, а не обычный shared-хостинг.
- ☐ DNS доменов `platform.example.ru` и `api.platform.example.ru` указывает на IP сервера.
- ☐ Файл `.env` заполнен production-значениями.
- ☐ Пароли PostgreSQL, MinIO/S3, admin и JWT secrets заменены на сильные.
- ☐ PostgreSQL, Redis и MinIO не открыты напрямую в интернет.
- ☐ `docker compose -f docker-compose.prod.yml ps` показывает запущенные сервисы.
- ☐ `/health` возвращает `database: "ok"` и `postgis: true`.
- ☐ SSL выпущен через Certbot для frontend и API доменов.
- ☐ Вход под admin работает.
- ☐ Открываются каталог, карта, карточка объекта, админка пользователей и объектов.
- ☐ Загрузка изображений и PDF работает.
- ☐ Если используется WordPress import, сначала проверен preview.

14. Спорные места и что улучшить позже

- Текущий `apps/web/Dockerfile` запускает `npm dev`. Для настоящего production позже лучше заменить это на сборку `npm build:web` и отдачу статических файлов через Nginx.
- Managed PostgreSQL Beget можно использовать только если подключено расширение `PostGIS`. Если расширение недоступно, оставляйте `postgis/postgis` в Docker.
- Beget S3 bucket нужно создать вручную в панели, потому что создание bucket через SDK у Beget ограничено.

15. Ссылки на документацию Beget

- Docker VPS: <https://beget.com/ru/cloud/marketplace/docker>
- VPS/VDS: <https://beget.com/ru/vps>
- S3 Beget: <https://beget.com/ru/kb/manual/obektnoe-hranilishche-s3-v-beget>
- PostgreSQL Beget: <https://beget.com/ru/kb/manual/cloud-postgresql>

Подготовлено для проекта Platforma. Перед выполнением команд внимательно замените все placeholder-значения.